

Controle sem modelo e aproximação de função valor

MC e TD em lote • ϵ -guloso e GLIE • SARSA e Q-learning • VFA • DQN

Prof. Marcelo Keese Albertini

Adaptado de CS234: Reinforcement Learning, Emma Brunskill (Stanford), Lecture 4. Leitura complementar: Sutton & Barto (2018), seções 5.2–5.4, 6.4–6.7 e 9.3–9.4.

Na Aula 3 aprendemos a *medir* uma política sem conhecer o modelo do mundo. Esta aula fecha esse assunto e passa ao problema difícil: *escolher* a política. O obstáculo novo é que, para decidir, o agente precisa de dados sobre ações que a política atual não toma — e é daí que nascem a exploração, o ϵ -guloso, o SARSA e o Q-learning. Na segunda metade abandonamos a tabela e passamos a representar Q por uma função parametrizada, o que abre a porta para o Deep Q-Learning e, junto com ela, para a instabilidade.

Intuição

Fio condutor. Todos os algoritmos desta aula são a mesma linha,

$$\text{estimativa} \leftarrow \text{estimativa} + \alpha (\text{alvo} - \text{estimativa}),$$

com escolhas diferentes de *alvo*. Quando um algoritmo novo aparecer, a primeira pergunta útil quase sempre é: *qual é o alvo, e ele é enviesado?*

Roteiro.

1. Avaliação de política em lote: o que MC e TD realmente calculam.
2. Melhoria de política generalizada: por que Q , e por que explorar.
3. Controle tabular sem modelo: MC-controle, GLIE, SARSA, Q-learning.
4. Aproximação de função valor.
5. Controle com aproximação e Deep Q-Learning.

1 Avaliação de política em lote

Nas versões vistas na Aula 3, cada tupla (s, a, r, s') era usada uma vez e descartada. No cenário **em lote** (*batch*, ou *offline*) temos um conjunto fixo de K episódios e podemos reprocessá-lo quantas vezes quisermos:

Algoritmo — Avaliação de política em lote

- 1: Dado o conjunto fixo de K episódios
- 2: **repita** até convergir
- 3: amostre um episódio $k \sim \{1, \dots, K\}$
- 4: aplique a atualização de MC (ou de TD(0)) a esse episódio
- 5: **fim**

A pergunta interessante é: rodando MC e TD(0) sobre *os mesmos* dados até a convergência, eles chegam à mesma resposta? A resposta é não — e entender por quê é entender a diferença conceitual entre os dois métodos.

1.1 O exemplo A–B

(Sutton & Barto, 2018, Exemplo 6.4.) Dois estados, A e B , com $\gamma = 1$. Observamos oito episódios:

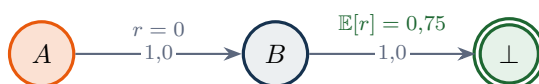
Episódio	Frequência
$A, 0, B, 0$	1 vez
$B, 1$	6 vezes
$B, 0$	1 vez

Note que A aparece **uma única vez** em toda a base. Partindo de B , houve recompensa 1 em 6 das 8 visitas, logo $V(B) = 6/8 = 0,75$ pelos dois métodos. Mas para $V(A)$:

$$V_{MC}(A) = 0 \quad \text{e} \quad V_{TD}(A) = 0,75$$

Monte Carlo. O único retorno já observado a partir de A foi $G = 0 + 0 = 0$. MC minimiza o erro quadrático em relação aos retornos *observados*, e responde 0.

TD(0). O alvo é $r + \gamma V(B) = 0,75$. TD encadeia a informação obtida em B , mesmo que A tenha sido visitado uma vez só. Implicitamente, TD construiu este MDP:



1.2 Para onde cada método converge

Monte Carlo em lote

Converge para os valores que minimizam o erro quadrático médio em relação aos retornos observados,

$$V = \arg \min_V \sum_{k=1}^K \sum_{t=1}^{L_k} (G_{k,t} - V(s_{k,t}))^2.$$

TD(0) em lote

Converge para o V^π do **MDP de máxima verossimilhança** estimado a partir dos dados — isto é, produz exatamente o mesmo resultado da programação dinâmica com *equivalência de certeza*:

$$\hat{P}(s' | s, a) = \frac{1}{N(s, a)} \sum_k \mathbf{1}\{s_k = s, a_k = a, s_{k+1} = s'\},$$

$$\hat{R}(s, a) = \frac{1}{N(s, a)} \sum_k \mathbf{1}\{s_k = s, a_k = a\} r_k.$$

Qual das duas respostas é “a certa” depende inteiramente de acreditarmos, ou não, na hipótese de Markov.

Intuição

Se o ambiente é markoviano, TD está certo: o que se aprende sobre B vale para *qualquer* caminho que leve a B , inclusive o que passa por A . TD usa a estrutura do problema; MC a ignora.

Atenção

Se o ambiente não é markoviano — estado parcialmente observado, variáveis latentes não registradas — TD propaga uma hipótese falsa. MC, que só olha retornos completos, é mais robusto, ao custo de variância.

Para discutir em aula

Em um problema de saúde, com prontuários incompletos e comorbidades não registradas, o estado observado quase certamente não é markoviano. Isso torna o TD inutilizável, ou apenas enviesado de forma controlável? Que evidência empírica você exigiria antes de decidir?

1.3 Critérios de comparação

	Monte Carlo	TD(0)	Eq. de certeza
Supõe Markov	não	sim	sim
Viés do alvo	não	sim	sim
Variância	alta	baixa	baixa
Precisa do episódio completo	sim	não	não
Custo por atualização	$O(1)$	$O(1)$	alto (sistema linear)
Eficiência de dados	baixa	média	alta

Em um episódio de comprimento L , tanto MC quanto TD(0) custam $O(L)$ por episódio — mas o TD já atualiza no primeiro passo, enquanto o MC precisa esperar o fim. A equivalência de certeza é a mais eficiente em dados e a mais cara em computação: há uma escada de eficiência estatística paga em tempo de processamento, e boa parte da pesquisa em RL sem modelo vive nessa tensão.

2 Melhoria de política generalizada

O esqueleto do controle é o mesmo da Aula 2:

Algoritmo — Iteração de política generalizada

- 1: Inicialize π
- 2: **repita**
- 3: *Avaliação*: estime Q^π
- 4: *Melhoria*: atualize π a partir de Q^π
- 5: **fim**

mas agora com três buracos: (i) se π é determinística, nunca observamos $Q(s, a)$ para $a \neq \pi(s)$; (ii) a melhoria usa um $\hat{Q}$ estimado, com ruído; (iii) não é óbvio quanto avaliar antes de melhorar.

2.1 Por que Q e não V

Com modelo, a melhoria gulosa se faz a partir de V :

$$\pi'(s) = \arg \max_{a \in \mathcal{A}} \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s') \right].$$

Sem modelo, R e P são desconhecidos e essa expressão é inútil. Já com Q :

$$\pi'(s) = \arg \max_{a \in \mathcal{A}} Q(s, a),$$

sem nenhum conhecimento da dinâmica.

Intuição

Q é exatamente V com a informação de um passo do modelo já embutida. É por isso que praticamente todo algoritmo de controle sem modelo aprende Q , e não V . O preço: a tabela cresce de $|\mathcal{S}|$ para $|\mathcal{S}||\mathcal{A}|$ entradas, e cada entrada precisa ser *visitada* para ser estimada.

2.2 O problema da exploração

- **Explorar:** não há como aprender sobre uma ação sem experimentá-la; dados sobre a só existem se π der probabilidade positiva a a .
- **Exploitar:** cada passo gasto experimentando é um passo não gasto colhendo a recompensa que já sabemos existir — e em domínios como saúde e educação, explorar tem custo real.

Atenção

Uma política **gulosa pura** sobre $\hat{Q}$ pode travar para sempre: basta que a primeira amostra de uma ação boa seja ruim por azar para que ela nunca mais seja escolhida — e, portanto, nunca mais corrigida.

Definição — Política ϵ -gulosa em relação a $Q(s, a)$

Seja $|\mathcal{A}|$ o número de ações e $\epsilon \in [0, 1]$. Então

$$\pi(a | s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|}, & \text{se } a = \arg \max_{a'} Q(s, a'), \\ \frac{\epsilon}{|\mathcal{A}|}, & \text{caso contrário.} \end{cases}$$

Isto é: com probabilidade $1 - \epsilon$ escolha a ação gulosa; com probabilidade ϵ escolha uniformemente ao acaso entre *todas* as ações.

Toda ação tem probabilidade pelo menos $\epsilon/|\mathcal{A}| > 0$, de modo que a exploração é garantida. Os casos extremos são $\epsilon = 1$ (passeio aleatório) e $\epsilon = 0$ (guloso puro). Uma alternativa comum é a política *softmax* (Boltzmann), $\pi(a | s) \propto e^{Q(s,a)/\tau}$, que explora preferencialmente as ações *quase* boas em vez de tratar todas as não-gulosas igualmente.

Atenção

Erro frequente: com $\epsilon = 1/3$ e $|\mathcal{A}| = 2$, a probabilidade de escolher a ação gulosa **não** é $2/3$, e sim $1 - \frac{1}{3} + \frac{1}{6} = \frac{5}{6}$ — a exploração uniforme também pode cair sobre a ação gulosa.

2.3 A melhoria monotônica sobrevive à exploração

Na Aula 2 provamos que a iteração de política melhora monotonicamente, mas a prova supunha que a melhoria produzisse uma política *determinística*. Uma política ϵ -gulosa é estocástica de propósito. A garantia se perde?

Teorema — melhoria monotônica ϵ -gulosa

Seja π_i uma política ϵ -gulosa qualquer e seja π_{i+1} a política ϵ -gulosa em relação a Q^{π_i} . Então $V^{\pi_{i+1}}(s) \geq V^{\pi_i}(s)$ para todo $s \in \mathcal{S}$.

Demonstração

$$\begin{aligned}
Q^{\pi_i}(s, \pi_{i+1}(s)) &= \sum_{a \in \mathcal{A}} \pi_{i+1}(a | s) Q^{\pi_i}(s, a) \\
&= \frac{\epsilon}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} Q^{\pi_i}(s, a) + (1 - \epsilon) \max_a Q^{\pi_i}(s, a) \\
&\geq \frac{\epsilon}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} Q^{\pi_i}(s, a) + (1 - \epsilon) \sum_{a \in \mathcal{A}} \underbrace{\frac{\pi_i(a | s) - \epsilon/|\mathcal{A}|}{1 - \epsilon}}_{w_a} Q^{\pi_i}(s, a) \\
&= \sum_{a \in \mathcal{A}} \pi_i(a | s) Q^{\pi_i}(s, a) = V^{\pi_i}(s).
\end{aligned}$$

A desigualdade vale porque o máximo é maior ou igual a *qualquer* combinação convexa. Os pesos w_a são não-negativos precisamente porque π_i é ϵ -gulosa, logo $\pi_i(a | s) \geq \epsilon/|\mathcal{A}|$, e somam 1 porque $\sum_a \pi_i(a | s) = 1$. De $Q^{\pi_i}(s, \pi_{i+1}(s)) \geq V^{\pi_i}(s)$ segue $V^{\pi_{i+1}} \geq V^{\pi_i}$ pelo teorema de melhoria de política da Aula 2. ■

Intuição

Podemos explorar *para sempre* e ainda assim nunca piorar. O custo é que a melhor política entre as ϵ -gulosas não é π^* — ela carrega o ϵ de ruído. É esse resíduo que a condição GLIE, a seguir, elimina.

3 Controle tabular sem modelo**3.1 Monte Carlo para Q**

A avaliação Monte Carlo da Aula 3, com o par (s, a) no lugar de s , e a melhoria feita a cada episódio:

Algoritmo — Controle MC *on-policy* com ϵ -guloso

- 1: Inicialize $Q(s, a) = 0$ e $N(s, a) = 0$ para todo (s, a) ; $\epsilon = 1$; $k = 1$
- 2: $\pi_k \leftarrow \epsilon$ -guloso(Q)
- 3: **repita**
- 4: gere o episódio k seguindo π_k e calcule os retornos $G_{k,t} = \sum_{j \geq t} \gamma^{j-t} r_{k,j}$
- 5: **para** $t = 1, \dots, T$ **faça**
- 6: **se** é a primeira visita a (s_t, a_t) no episódio k **então**
- 7: $N(s_t, a_t) += 1$
- 8: $Q(s_t, a_t) += \frac{1}{N(s_t, a_t)} (G_{k,t} - Q(s_t, a_t))$
- 9: **fim**
- 10: **fim**
- 11: $k += 1$; $\epsilon \leftarrow 1/k$ (*exploração decrescente*)
- 12: $\pi_k \leftarrow \epsilon$ -guloso(Q) (*melhoria de política*)
- 13: **fim**

A melhoria acontece a *cada* episódio, sem esperar a avaliação convergir: é a versão impaciente da iteração de política — e ainda assim funciona.

Exercício 1 — MC-controle no Mars Rover

Sete estados, duas ações, $\gamma = 1$, com $R(\cdot, a_1) = [1 \ 0 \ 0 \ 0 \ 0 \ 0 \ +10]$ e $R(\cdot, a_2) = [0 \ 0 \ 0 \ 0 \ 0 \ 0 \ +5]$. Comece com $Q(s, a) = 0$ para todo par, $\pi(s) = a_1$ para todo s e $\epsilon = 0,5$.

(continua)

Amostrou-se a trajetória

$$\tau = (s_3, a_1, 0, s_2, a_2, 0, s_3, a_1, 0, s_2, a_2, 0, s_1, a_1, 1, \perp).$$

(a) Calcule a estimativa MC de primeira visita de Q para cada par. **(b)** Qual passa a ser a política gulosa? **(c)** Com $\epsilon = 1/3$, qual a probabilidade de escolher a_1 em s_1 ?

Resposta. (a) $Q(\cdot, a_1) = [1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0]$ e $Q(\cdot, a_2) = [0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0]$: toda primeira visita ocorre antes da recompensa 1 recebida em s_1 e, com $\gamma = 1$, cada par visitado recebe retorno 1. (b) $\pi = [a_1 \ a_2 \ a_1 \ \text{empate} \ \dots]$. (c) $5/6$.

3.2 GLIE

Definição — GLIE — Greedy in the Limit with Infinite Exploration

Uma sequência de políticas $\{\pi_i\}$ é GLIE se

1. todo par estado-ação é visitado infinitas vezes, $\lim_{i \rightarrow \infty} N_i(s, a) \rightarrow \infty$;
2. a política de comportamento converge para a gulosa, $\lim_{i \rightarrow \infty} \pi_i(a | s) \rightarrow \arg \max_a Q(s, a)$ com probabilidade 1.

As duas condições puxam em direções opostas: a primeira exige exploração eterna, a segunda exige que ela desapareça. A estratégia GLIE mais simples concilia ambas: ϵ -guloso com $\epsilon_i = 1/i$, pois $\sum_i 1/i$ diverge (exploração infinita) mas $\epsilon_i \rightarrow 0$ (guloso no limite).

Teorema — convergência do controle MC com GLIE

O controle Monte Carlo GLIE converge para a função valor de ação ótima: $Q(s, a) \rightarrow Q^*(s, a)$.

3.3 On-policy e off-policy

Definição

Aprendizado on-policy estima e melhora a política a partir da experiência gerada por ela mesma: a política que age é a política que se aprende (MC-controle, SARSA).

Aprendizado off-policy estima e melhora uma política *alvo* usando dados coletados por uma política de *comportamento* diferente, π_b (Q-learning, DQN, praticamente todo RL *offline* moderno).

Off-policy é o que permite aprender de dados de arquivo — registros médicos, logs de recomendação, demonstrações humanas — e reutilizar experiência antiga, que é a base do *replay* do DQN. Em troca, é a fonte de boa parte da instabilidade que aparece quando se combina off-policy com aproximação de função.

3.4 SARSA e Q-learning

O nome SARSA vem da quintupla usada na atualização, (s, a, r, s', a') .

Definição — Atualização SARSA

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(\underbrace{r_t + \gamma Q(s_{t+1}, a_{t+1})}_{\text{alvo: a ação realmente tomada}} - Q(s_t, a_t) \right)$$

Definição — Atualização Q-learning (Watkins, 1989)

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(\underbrace{r_t + \gamma \max_{a'} Q(s_{t+1}, a')}_{\text{alvo: a melhor ação disponível}} - Q(s_t, a_t) \right)$$

Algoritmo — SARSA com exploração ϵ -gulosa

- 1: Inicialize $Q(s, a)$; $t = 0$; estado inicial s_0 ; $\pi \leftarrow \epsilon$ -guloso(Q)
- 2: Tome $a_t \sim \pi(s_t)$
- 3: **repita**
- 4: observe (r_t, s_{t+1}) e tome $a_{t+1} \sim \pi(s_{t+1})$
- 5: $Q(s_t, a_t) += \alpha(r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t))$
- 6: $\pi \leftarrow \epsilon$ -guloso(Q); $t += 1$
- 7: **fim**

Algoritmo — Q-learning com exploração ϵ -gulosa

- 1: Inicialize $Q(s, a)$ para todo $s \in \mathcal{S}$, $a \in \mathcal{A}$; $t = 0$; estado inicial s_0
- 2: Defina $\pi_b \leftarrow \epsilon$ -guloso(Q) (*política de comportamento*)
- 3: **repita**
- 4: tome $a_t \sim \pi_b(s_t)$ e observe (r_t, s_{t+1})
- 5: $Q(s_t, a_t) += \alpha(r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t))$
- 6: $\pi_b \leftarrow \epsilon$ -guloso(Q); $t += 1$
- 7: **fim**

Intuição

SARSA avalia a política ϵ -gulosa *incluindo o custo de explorar*: aprende o valor de ser um agente que às vezes erra de propósito. O max do Q-learning desacopla o que se *aprende* do que se *faz* — é a versão amostral do operador de backup de Bellman **ótimo** da Aula 2, no lugar do operador de avaliação.

Teorema — convergência do Q-learning tabular

Para MDPs com $\mathcal{S}$ e $\mathcal{A}$ finitos, o Q-learning converge para $Q(s, a) \rightarrow Q^*(s, a)$ desde que (i) a sequência de políticas satisfaça a condição GLIE e (ii) os passos α_t satisfaçam as condições de Robbins–Monro:

$$\sum_{t=1}^{\infty} \alpha_t = \infty, \quad \sum_{t=1}^{\infty} \alpha_t^2 < \infty.$$

A primeira soma garante que qualquer erro inicial *pode* ser corrigido (os passos não somem cedo demais); a segunda garante que o ruído das amostras é eventualmente amortecido. Por exemplo, $\alpha_t = 1/t$ satisfaz ambas. A demonstração se apoia em aproximação estocástica, na propriedade de contração do backup de Bellman (Aula 2) e em recompensas e valores limitados.

Atenção

Nenhuma dessas três hipóteses sobrevive intacta quando trocamos a tabela por um aproximador de função — guarde essa observação para a Seção 5.

3.5 A diferença em um único passo

Mars Rover com $\alpha = 0,5$, $\gamma = 1$, $Q(\cdot, a_1) = [1 \ 0 \ 0 \ 0 \ 0 \ 10]$ e $Q(\cdot, a_2) = [1 \ 0 \ 0 \ 0 \ 0 \ 5]$. O agente parte de s_6 , toma a_1 , recebe $r = 0$ e vai para s_7 , onde toma a_2 .

SARSA — tupla $(s_6, a_1, 0, s_7, a_2)$	$Q(s_6, a_1) \leftarrow 0 + 0,5 (0 + 1 \cdot \underbrace{Q(s_7, a_2)}_{5}) = 2,5$
Q-learning — tupla $(s_6, a_1, 0, s_7)$	$Q(s_6, a_1) \leftarrow 0 + 0,5 (0 + 1 \cdot \underbrace{\max_{a'} Q(s_7, a')}_{10}) = 5$

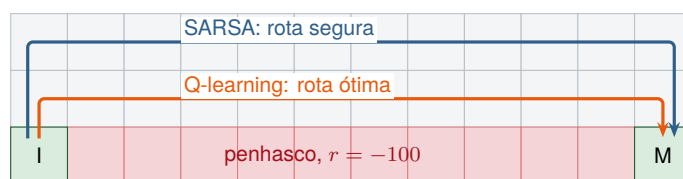
Mesma transição, dois números. O SARSA usou o valor da ação de fato tomada em s_7 , possivelmente escolhida por exploração; o Q-learning usou o máximo, $\max\{10, 5\} = 10$, ignorando qual ação o agente escolheu.

Atenção

A inicialização de Q importa? **Assintoticamente não**, sob as condições do teorema de convergência — mas **muito** no início, e é ela que produz a diferença 2,5 contra 5 acima. Inicializar otimisticamente é, aliás, uma estratégia de exploração por si só.

3.6 O que essa diferença provoca: *cliff walking*

(Sutton & Barto, 2018, Exemplo 6.6.) Grade 4×12 , recompensa -1 por passo; cair no penhasco custa -100 e devolve o agente ao início.



O **Q-learning** aprende o caminho ótimo, rente ao penhasco — mas, agindo com ϵ -guloso, cai nele de vez em quando e obtém *recompensa média pior*. O **SARSA** aprende o caminho seguro, pela rota de cima, porque seu alvo já contabiliza os passos exploratórios. Se $\epsilon \rightarrow 0$, ambos convergem para a política ótima; mas *durante* o aprendizado o comportamento cauteloso do SARSA é frequentemente preferível — especialmente se o “penhasco” for real.

Para discutir em aula

Em que aplicações o critério relevante é a política final, e em quais é o desempenho *durante* o aprendizado? O que isso implica sobre escolher entre SARSA e Q-learning num sistema que aprende em produção?

3.7 Um viés escondido no max

O alvo do Q-learning usa $\max_{a'} \hat{Q}(s', a')$ como estimativa de $\max_{a'} Q(s', a')$. Mas o máximo de estimativas ruidosas é um estimador **enviesado para cima**: pela desigualdade de Jensen, como o max é uma função convexa,

$$\mathbb{E} \left[\max_{a'} \hat{Q}(s', a') \right] \geq \max_{a'} \mathbb{E} [\hat{Q}(s', a')].$$

Basta que uma ação tenha tido sorte nas amostras para que o max a selecione, e o erro é então propagado adiante pelo *bootstrapping*. Esse **viés de maximização** é sistemático, não ruído: não desaparece com mais estados e piora quanto maior for $|\mathcal{A}|$.

Intuição

A correção clássica é **desacoplar seleção e avaliação** da ação: usar um conjunto de estimativas para escolher o $\arg \max$ e outro para avaliá-lo. É a ideia do *Double Q-learning* e, depois, do *Double DQN*.

4 Aproximação de função valor

4.1 Por que a tabela não basta

Domínio	$ S $ aproximado
<i>Mars Rover</i> (Aula 2)	7
Gamão	10^{20}
Go (19×19)	10^{170}
Atari (4 quadros 84×84)	256^{28224}
Robô com estados contínuos	incontável

Não é apenas uma questão de memória: mesmo que a tabela coubesse, seria preciso *visitar* cada entrada ao menos uma vez para estimá-la. Queremos evitar armazenar por estado o modelo (P, R) , o valor V , o valor de ação Q e a própria política π — reduzindo memória, computação e, sobretudo, a **experiência** necessária.

Intuição

A palavra-chave é **generalização**: aprender sobre um estado e, com isso, saber algo sobre estados parecidos que nunca foram visitados.

4.2 O problema, primeiro com um oráculo

Suponha que exista um oráculo capaz de devolver o valor verdadeiro $Q^\pi(s, a)$ para qualquer par consultado. Então temos um problema de regressão comum sobre pares $((s, a), Q^\pi(s, a))$, com perda

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[(Q^\pi(s, a) - \hat{Q}(s, a; \mathbf{w}))^2 \right].$$

Como

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = -2 \mathbb{E}_\pi \left[(Q^\pi(s, a) - \hat{Q}(s, a; \mathbf{w})) \nabla_{\mathbf{w}} \hat{Q}(s, a; \mathbf{w}) \right],$$

o passo $\Delta \mathbf{w} = -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w})$, aplicado a uma única amostra, fica

$$\Delta \mathbf{w} = \alpha (Q^\pi(s, a) - \hat{Q}(s, a; \mathbf{w})) \nabla_{\mathbf{w}} \hat{Q}(s, a; \mathbf{w}).$$

O gradiente estocástico usa uma (ou poucas) amostras no lugar da esperança; em média, o passo é o mesmo do gradiente completo.

4.3 Aproximadores lineares e o caso tabular

Descreva cada par por um vetor de atributos $\mathbf{x}(s, a) = [x_1(s, a), \dots, x_d(s, a)]^\top$ e tome

$$\hat{Q}(s, a; \mathbf{w}) = \mathbf{w}^\top \mathbf{x}(s, a), \quad \nabla_{\mathbf{w}} \hat{Q}(s, a; \mathbf{w}) = \mathbf{x}(s, a),$$

de modo que a atualização fica $\Delta \mathbf{w} = \alpha (\text{alvo} - \mathbf{w}^\top \mathbf{x}(s, a)) \mathbf{x}(s, a)$.

Definição — O tabular como caso especial

Tome $d = |\mathcal{S}||\mathcal{A}|$ e atributos *one-hot*, $x_i(s, a) = \mathbf{1}\{(s, a) = i\}$. Então $\mathbf{w}$ é a tabela, $\nabla_{\mathbf{w}}\hat{Q}$ seleciona uma única coordenada, e a atualização acima reduz-se exatamente ao Q-learning tabular. Não há dois algoritmos: há um só, com representações diferentes — e no caso one-hot simplesmente não existe generalização, o que é também a razão de nada poder ser estragado pela aproximação.

4.4 Sem oráculo: substituir o alvo

O retorno G_t é uma amostra **não-enviesada**, ainda que ruidosa, de $Q^\pi(s_t, a_t)$. Basta então substituir o oráculo por G_t e fazer regressão supervisionada sobre os pares $\langle (s_t, a_t), G_t \rangle$. Com aproximação linear, isso converge para o mínimo global da perda quadrática — a única combinação desta aula com uma garantia tão confortável.

Algoritmo — Avaliação de política MC com aproximação de função

```

1: Inicialize  $\mathbf{w}$ ;  $k = 1$ 
2: repita
3:   gere o episódio  $k$  seguindo  $\pi$ 
4:   para  $t = 1, \dots, L_k$  faça
5:     se é a primeira visita a  $(s_t, a_t)$  então
6:        $G_t = \sum_{j=t}^{L_k} \gamma^{j-t} r_{k,j}$ 
7:        $\mathbf{w} += \alpha (G_t - \hat{Q}(s_t, a_t; \mathbf{w})) \nabla_{\mathbf{w}} \hat{Q}(s_t, a_t; \mathbf{w})$ 
8:     fim
9:   fim;  $k += 1$ 
10: fim
```

Com TD(0), o alvo passa a ser $r + \gamma \hat{V}^\pi(s'; \mathbf{w})$: enviesado e aproximado. São três aproximações empilhadas:

Atenção

- 1. Amostragem** — usamos uma transição no lugar da esperança sobre P ;
- 2. Bootstrapping** — usamos nossa própria estimativa no lugar do valor verdadeiro;
- 3. Aproximação de função** — projetamos o resultado numa família paramétrica que talvez não contenha V^π .

Algoritmo — Avaliação de política TD(0) com aproximação de função

```

1: Inicialize  $\mathbf{w}$  e o estado  $s$ 
2: repita
3:   amostre  $a \sim \pi(s)$ ; observe  $r(s, a)$  e  $s' \sim P(\cdot | s, a)$ 
4:    $\mathbf{w} += \alpha (r + \gamma \hat{V}(s'; \mathbf{w}) - \hat{V}(s; \mathbf{w})) \nabla_{\mathbf{w}} \hat{V}(s; \mathbf{w})$ 
5:   se  $s'$  não é terminal então  $s \leftarrow s'$  senão reinicie o episódio
6: fim
```

Intuição

Semigradiante. O termo $\gamma \hat{V}(s'; \mathbf{w})$ também depende de $\mathbf{w}$, mas o tratamos como constante ao derivar — logo a atualização acima não é o gradiente verdadeiro de J . Derivar os dois lados produziria o *gradiente residual*: estável, porém lento e exigindo duas amostras independentes do próximo estado. O semigradiante aprende mais rápido, ao custo das garantias, e é o que

(continua)

se usa na prática.

5 Controle com aproximação e Deep Q-Learning

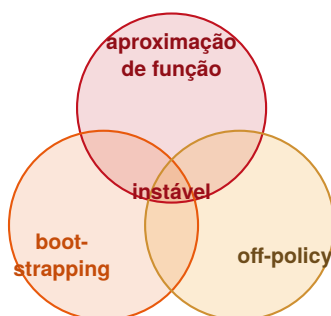
5.1 Uma linha de código, três algoritmos

Representamos os valores de ação por $\hat{Q}^\pi(s, a; \mathbf{w}) \approx Q^\pi$ e intercalamos avaliação aproximada com melhoria ϵ -gulosa. Como $Q^\pi(s_t, a_t)$ é desconhecido, substituímos por um **alvo**:

Molde geral	$\Delta \mathbf{w} = \alpha (\text{alvo} - \hat{Q}(s_t, a_t; \mathbf{w})) \nabla_{\mathbf{w}} \hat{Q}(s_t, a_t; \mathbf{w})$
Monte Carlo	alvo = G_t
SARSA	alvo = $r + \gamma \hat{Q}(s', a'; \mathbf{w})$
Q-learning	alvo = $r + \gamma \max_{a'} \hat{Q}(s', a'; \mathbf{w})$
DQN	alvo = $r + \gamma \max_{a'} \hat{Q}(s', a'; \mathbf{w}^-)$, com (s, a, r, s') do <i>buffer</i>

Toda a Seção 3 reaparece aqui trocando “entrada da tabela” por “saída da rede”.

5.2 A tríade mortal



Cada atualização faz um backup de Bellman aproximado e em seguida *projeta* o resultado sobre a família de funções escolhida. O backup de Bellman é uma **contração**, mas a projeção pode ser uma **expansão**: a composição pode não contrair, e aí a sequência oscila ou diverge. Dois a dois, os três elementos são seguros; os três juntos, não.

Atenção

O contraexemplo canônico é o de **Baird** (Sutton & Barto, 2018), em que o Q-learning com aproximação *linear* diverge, com os pesos crescendo sem limite, num MDP de apenas sete estados.

	Algoritmo	Tabular	Linear	Não-linear
Avaliação	Monte Carlo	converge	converge	converge
	TD(0) on-policy	converge	converge [†]	pode divergir
	TD(0) off-policy	converge	pode divergir	pode divergir
Controle	MC-controle	converge	oscila próximo	pode divergir
	SARSA	converge	oscila próximo	pode divergir
	Q-learning	converge	pode divergir	pode divergir

[†] para um ponto fixo cujo erro é limitado por $\frac{1}{1-\gamma}$ vezes o erro da melhor aproximação possível na família.

Intuição

Note onde está o Q-learning com rede neural: na célula sem garantia alguma. O DQN não resolve isso teoricamente — ele *engenheira* o problema para que, na prática, funcione.

5.3 As duas ideias do DQN

Além da tríade, dois problemas concretos aparecem ao treinar $\hat{Q}$ com uma rede neural.

- 1. Correlação entre amostras.** Transições consecutivas são fortemente dependentes, e o SGD supõe amostras aproximadamente i.i.d.
- 2. Alvos não-estacionários.** O alvo $r + \gamma \max_{a'} \hat{Q}(s', a'; \mathbf{w})$ usa os mesmos pesos que estamos atualizando: perseguimos um alvo que se move a cada passo.

Definição — Repetição de experiência (*experience replay*)

Mantenha um *buffer* $\mathcal{D}$ com as tuplas (s, a, r, s') já vividas (tipicamente 10^6 transições, em janela deslizante) e treine amostrando *minibatches* aleatórios dele, em vez de usar a transição mais recente.

Isso só é possível porque o Q-learning é **off-policy**: as tuplas do *buffer* foram geradas por políticas antigas, e isso não invalida o alvo, que já usa o max e não a ação realmente escolhida. O SARSA não pode fazer o mesmo sem correção.

Definição — Alvos fixos (*fixed Q-targets*)

Mantenha dois conjuntos de pesos: $\mathbf{w}$, os que estão sendo atualizados, e $\mathbf{w}^-$, os de uma *rede-alvo* congelada. O alvo passa a ser calculado com $\mathbf{w}^-$,

$$\Delta \mathbf{w} = \alpha \left(r + \gamma \max_{a'} \hat{Q}(s', a'; \mathbf{w}^-) - \hat{Q}(s, a; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s, a; \mathbf{w}),$$

e a cada C passos sincroniza-se $\mathbf{w}^- \leftarrow \mathbf{w}$.

Intuição

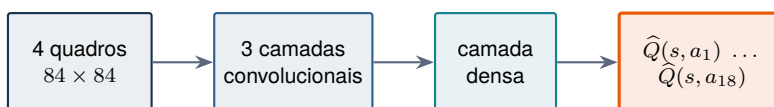
Entre duas sincronizações, o problema vira uma regressão supervisionada comum, com rótulos fixos. É a manobra que devolve estabilidade a um procedimento que, de outro modo, persegue a própria cauda.

Algoritmo — Deep Q-Network (Mnih et al., 2015)

- 1: Entrada: $C, \alpha, \text{buffer } \mathcal{D} = \{\}$; inicialize $\mathbf{w}, \mathbf{w}^- \leftarrow \mathbf{w}, t = 0$; obtenha s_0
- 2: **repita**
- 3: amostre a_t pela política ϵ -gulosa sobre $\hat{Q}(s_t, \cdot; \mathbf{w})$
- 4: observe r_t e s_{t+1} ; armazene (s_t, a_t, r_t, s_{t+1}) em $\mathcal{D}$
- 5: amostre um *minibatch* aleatório $\{(s_i, a_i, r_i, s_{i+1})\}$ de $\mathcal{D}$
- 6: **para** cada i no *minibatch* **faça**
- 7: $y_i = r_i$ se o episódio terminou em $i + 1$; $y_i = r_i + \gamma \max_{a'} \hat{Q}(s_{i+1}, a'; \mathbf{w}^-)$ caso contrário
- 8: passo de descida em $(y_i - \hat{Q}(s_i, a_i; \mathbf{w}))^2$: $\mathbf{w} += \alpha (y_i - \hat{Q}(s_i, a_i; \mathbf{w})) \nabla_{\mathbf{w}} \hat{Q}(s_i, a_i; \mathbf{w})$
- 9: **fim**; $t += 1$
- 10: **se** $t \bmod C = 0$ **então** $\mathbf{w}^- \leftarrow \mathbf{w}$
- 11: **fim**

Atenção

A rede-alvo **dobra o requisito de memória** (dois conjuntos de pesos), mas **não** dobra o tempo de computação: em ambos os casos há uma passagem para frente (para o alvo) e uma para frente/para trás (para a atualização). O extra é apenas a cópia $\mathbf{w}^- \leftarrow \mathbf{w}$ a cada C passos.

5.4 DQN em Atari

O estado s é a pilha dos quatro últimos quadros de pixels brutos — necessária para recuperar velocidade e direção, que um quadro único não revela. A saída é um valor $\hat{Q}(s, a)$ para cada uma das 18 posições de joystick, o que permite calcular $\max_{a'} \hat{Q}(s', a')$ com uma única passagem pela rede. A recompensa é a variação de pontuação no passo, recortada em $[-1, 1]$. A mesma arquitetura e os mesmos hiperparâmetros foram usados em todos os jogos.

Jogo	Linear	Rede profunda	DQN com alvo fixo	DQN com <i>replay</i>	DQN com <i>replay</i> e alvo fixo
Breakout	3	3	10	241	317
Enduro	62	29	141	831	1006
River Raid	2345	1453	2868	4102	7447
Seaquest	656	275	1003	823	2894
Space Invaders	301	302	373	826	1089

Trocar o modelo linear por uma rede profunda, **sozinho**, chegou a *piorar* o desempenho (Enduro: $62 \rightarrow 29$; Seaquest: $656 \rightarrow 275$). O que fez a diferença foram o *replay* e o alvo fixo — e o *replay* é, de longe, o fator mais decisivo. Por três razões:

1. **descorrelaciona** as amostras dentro do *minibatch*, aproximando a hipótese i.i.d. que o SGD supõe;
2. **reaproveita** cada transição muitas vezes — ganho enorme de eficiência de dados, que é o recurso caro quando cada amostra exige uma interação com o ambiente;
3. **estabiliza a distribuição de treino**: sem *buffer*, uma mudança de política altera a distribuição dos dados de imediato, e a rede “esquece” o que aprendeu nas regiões que deixou de visitar — o esquecimento catastrófico.

Para discutir em aula

O terceiro efeito costuma ser o mais subestimado. Que experimento você desenharia para separar, empiricamente, a contribuição da descorrelação da contribuição do reaproveitamento de dados?

5.5 Aperfeiçoamentos imediatos

- **Double DQN** (van Hasselt et al., AAAI 2016) — corrige o viés de maximização: a rede em treinamento *escolhe* a ação e a rede-alvo *avalia*,

$$y = r + \gamma \hat{Q}(s', \arg \max_{a'} \hat{Q}(s', a'; \mathbf{w}); \mathbf{w}^-).$$

- ▶ **Repetição priorizada** (Schaul et al., ICLR 2016) — amostrar do *buffer* com probabilidade proporcional ao erro TD, em vez de uniformemente: aprender mais onde ainda se erra mais.
- ▶ **Dueling DQN** (Wang et al., ICML 2016) — decompor $\hat{Q}(s, a) = \hat{V}(s) + \hat{A}(s, a)$ em valor de estado e vantagem, aprendendo $\hat{V}(s)$ mesmo quando a escolha da ação é irrelevante naquele estado.

As três extensões são ortogonais e podem ser combinadas — é essencialmente o que faz o *Rainbow* (Hessel et al., 2018), que soma seis delas.

6 Síntese

Algoritmo	Alvo	Tipo	Converge para
MC-controle (GLIE)	G_t	on-policy	Q^* (tabular)
SARSA	$r + \gamma Q(s', a')$	on-policy	Q^* (tabular, GLIE)
Q-learning	$r + \gamma \max_{a'} Q(s', a')$	off-policy	Q^* (tabular, GLIE + R.-M.)
MC com VFA	G_t	on-policy	mínimo global (linear)
SARSA com VFA	$r + \gamma \hat{Q}(s', a'; \mathbf{w})$	on-policy	oscila próximo (linear)
Q-learning com VFA	$r + \gamma \max_{a'} \hat{Q}(s', a'; \mathbf{w})$	off-policy	sem garantia
DQN	$r + \gamma \max_{a'} \hat{Q}(s', a'; \mathbf{w}^-)$	off-policy	sem garantia

Definição — O que você deve ser capaz de fazer

- ▶ Implementar avaliação de política por MC e TD(0), tabular e com aproximação de função.
- ▶ Implementar controle por MC, SARSA e Q-learning, e explicar exatamente onde os alvos de SARSA e Q-learning divergem.
- ▶ Enunciar as condições GLIE e Robbins–Monro e dizer para que serve cada uma.
- ▶ Listar os três elementos da tríade mortal e descrever qualitativamente o problema que geram.
- ▶ Explicar as duas ideias que tornaram o DQN viável: repetição de experiência e alvos fixos.

7 Exercícios

Exercício 2 — Exemplo A–B com desconto

Refaça o cálculo de $V(A)$ e $V(B)$ do exemplo A–B com $\gamma = 0,9$, para MC em lote e para TD(0) em lote. As duas respostas continuam diferindo? Em quanto? O que acontece com a diferença quando $\gamma \rightarrow 0$?

Exercício 3 — Os pesos da prova

Na demonstração da melhoria monotônica ϵ -gulosa, verifique explicitamente que os pesos $w_a = (\pi_i(a | s) - \epsilon/|\mathcal{A}|)/(1 - \epsilon)$ são não-negativos e somam 1. Onde exatamente a demonstração usa a hipótese de que π_i é ϵ -gulosa, e não uma política estocástica qualquer?

Exercício 4 — SARSA e Q-learning coincidem?

(a) Mostre que, se $\epsilon = 0$ em todos os instantes, a atualização de SARSA coincide com a de Q-learning, qualquer que seja a inicialização de Q . (b) Por que isso não é um bom argumento para usar $\epsilon = 0$? (c) Ambos podem alterar a política após cada passo. Verdadeiro ou falso,

(continua)

e por quê?

Exercício 5 — Tabular é linear

Escreva explicitamente os atributos *one-hot* $\mathbf{x}(s, a)$ e mostre, passo a passo, que $\Delta \mathbf{w} = \alpha(\text{alvo} - \mathbf{w}^\top \mathbf{x})\mathbf{x}$ reproduz a atualização tabular do Q-learning. Em seguida, explique por que a mesma conta *não* vale para atributos com sobreposição entre estados.

Exercício 6 — Viés de maximização

Considere um estado s' com duas ações cujos valores verdadeiros são ambos $Q(s', a_1) = Q(s', a_2) = 0$, mas cujas estimativas são independentes com distribuição $\mathcal{N}(0, 1)$. **(a)** Calcule $\mathbb{E}[\max\{\hat{Q}_1, \hat{Q}_2\}]$. **(b)** Compare com $\max\{\mathbb{E}[\hat{Q}_1], \mathbb{E}[\hat{Q}_2]\}$. **(c)** Como o Double Q-learning reduz esse viés? **(d)** O que acontece com o viés quando $|\mathcal{A}|$ cresce?

Exercício 7 — Replay e off-policy

(a) Explique por que o *replay* é compatível com o Q-learning mas não com o SARSA, sem correção. **(b)** Que correção seria necessária para usar um *buffer* com SARSA? **(c)** Um *buffer* muito grande pode prejudicar o aprendizado. Em que situação, e por quê?

Exercício 8 — Cliff walking na prática

Implemente SARSA e Q-learning no ambiente *cliff walking* da Seção 3.6, com $\alpha = 0,5$, $\gamma = 1$ e $\epsilon = 0,1$, por 500 episódios. **(a)** Registre o retorno médio por episódio de cada um. **(b)** Desenhe as políticas gulosas finais. **(c)** Repita com $\epsilon = 0,01$ e explique a mudança. **(d)** Qual dos dois você colocaria num sistema real que aprende em produção, e sob que condição mudaria de ideia?

8 Leituras

- ▶ Sutton & Barto, *Reinforcement Learning: An Introduction*, 2ª ed., 2018: seções 5.2–5.4 (controle Monte Carlo), 6.4–6.5 (SARSA e Q-learning), 6.7 (viés de maximização e Double Q-learning), 9.3–9.4 e 11.3 (aproximação de função e a tríade mortal).
- ▶ Watkins & Dayan, *Q-learning*, *Machine Learning*, 1992 — a prova de convergência original.
- ▶ Mnih et al., *Human-level control through deep reinforcement learning*, *Nature* 518, 2015 — o artigo do DQN, com o estudo de ablação reproduzido acima.
- ▶ van Hasselt, Guez & Silver, *Deep Reinforcement Learning with Double Q-learning*, AAAI 2016.
- ▶ Emma Brunskill, CS234 (Stanford), Lecture 4 — material de origem desta aula.